



UNIVERSIDADE
FEDERAL DO CEARÁ

Code smells em frameworks front-end

Disciplina: Qualidade de Software

Equipe: Luiz Eduardo Paiva Ribeiro; Luis Eduardo Vieira de Oliveira;

Projeto: <https://github.com/ghostfolio/ghostfolio>

Fork: <https://github.com/eduardoribeir/ghostfolio/tree/refatoracao-eduardo>

Tabela de code smells detectados:

https://docs.google.com/document/d/1CIBR3Pw_FPPaor2KkQeq40Ar-HM5qkgVel97gTZ-JUE/edit?usp=sharing

Eu (Luiz E. Paiva) estou atualmente trabalhando na refatoração do seguinte code smell:

Long Class (LC)

Minhas principais dificuldades na remoção do code smell são:

- Identificar quais responsabilidades poderiam ser extraídas sem quebrar a funcionalidade existente
- Decidir como organizar os novos serviços e interfaces de forma que fizesse sentido para o domínio
- Manter a compatibilidade com os templates existentes ao reestruturar os inputs dos componentes
- Entender todas as dependências entre os métodos antes de separar as responsabilidades
- Garantir que a separação não criaria acoplamento excessivo entre as novas classes

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

- Extract Class: Separei responsabilidades em novos serviços especializados
- Extract Interface: Criei interfaces para agrupar inputs relacionados (config/data/options)
- Move Method: Movi métodos de negócio dos componentes para os serviços
- Introduce Parameter Object: Agrupei parâmetros relacionados em objetos de configuração

Eu (Luiz E. Paiva) estou atualmente trabalhando na refatoração do seguinte code smell:

Too Much Information (TMI)

Minhas principais dificuldades na remoção do code smell são:

- Decidir quais detalhes internos realmente precisavam ser expostos publicamente
- Criar getters que encapsulassem a lógica sem expor demais a implementação interna
- Manter a retrocompatibilidade com templates que já usavam as propriedades antigas
- Balancear entre encapsulamento e facilidade de uso para outros desenvolvedores



UNIVERSIDADE FEDERAL DO CEARÁ

- Garantir que o encapsulamento não dificultasse os testes unitários

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

- Encapsulate Field: Transformei campos públicos em propriedades privadas com getters
- Hide Delegate: Criei interfaces que escondem detalhes de implementação
- Remove Setting Method: Removi setters desnecessários que expunham demais o estado interno
- Introduce Parameter Object: Usei objetos de configuração para limitar exposição de detalhes

Eu (Luis E. Vieira) estou atualmente trabalhando na refatoração do seguinte code smell:

Any

Minhas principais dificuldades na remoção do code smell são:

- Identificar onde o uso de 'any' escondia problemas reais de tipagem
- Entender corretamente o formato dos dados trafegados entre componentes e serviços
- Substituir 'any' por tipos mais específicos sem quebrar funcionalidades existentes
- Manter compatibilidade com templates que dependiam de propriedades não tipadas
- Evitar aumentar excessivamente a complexidade criando muitos tipos desnecessários

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

- Replace Type Code with Class: Substituí usos de 'any' por interfaces e tipos específicos
- Extract Interface: Criei interfaces para representar estruturas de dados reutilizáveis
- Introduce Parameter Object: Agrupei dados relacionados em objetos tipados
- Encapsulate Field: Reduzi o acesso direto a estruturas genéricas
- Rename Method / Variable: Ajustei nomes para refletir melhor os novos tipos definidos

Eu (Luis E. Vieira) estou atualmente trabalhando na refatoração do seguinte code smell:

Large File (LF)

Minhas principais dificuldades na remoção do code smell são:

- Identificar agrupamentos lógicos de funcionalidades dentro do arquivo grande sem quebrar a coesão



UNIVERSIDADE FEDERAL DO CEARÁ

- Decidir quais dependências devem ser mantidas no arquivo principal e quais podem ser movidas para arquivos separados
- Manter a compatibilidade de imports entre os arquivos recém-criados e os consumidores existentes
- Garantir que a divisão não crie dependências cíclicas entre os novos arquivos
- Preservar a visibilidade total da funcionalidade sem fragmentar demais a lógica de negócio
- Lidar com dependências compartilhadas entre as diferentes seções do arquivo grande

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

- Extract Class: Extraí classes relacionadas para arquivos separados baseado em responsabilidades comuns
- Move Method: Movi métodos agrupados por domínio para classes especializadas
- Group Related Logic: Agrupei funcionalidades relacionadas em módulos coesos
- Create Service Layers: Criei camadas de serviço para separar responsabilidades de negócio
- Modularize by Feature: Organizei os arquivos por funcionalidade em vez de por tipo
- Introduce Facade Pattern: Usei fachadas para manter interfaces simples enquanto a complexidade é movida para arquivos internos
- Separate Concerns: Separei claramente responsabilidades de controller, service e data access
- Create Helper Functions: Extraí funções utilitárias em arquivos dedicados